



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ
КАФЕДРА

Информатика и системы управления
Теоретическая информатика и компьютерные технологии

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К КУРСОВОЙ РАБОТЕ ПО ДИСЦИПЛИНЕ «АЛГОРИТМЫ КОМПЬЮТЕРНОЙ ГРАФИКИ» НА ТЕМУ:

Визуализация течения жидкости с
использованием WebGPU

Студент

ИУ9-52Б
(группа)

(подпись, дата)

Федуков А.А.
(И.О. Фамилия)

Руководитель курсовой работы

(подпись, дата)

Царев А.С.
(И.О. Фамилия)

Оценка

2025 г.

Содержание

Введение	4
1 Изучение предметной области	6
1.1 Классификация по способу представления среды	6
1.1.1 Эйлеровы (сеточные) методы	6
1.1.2 Лагранжевы (частичные) методы	6
1.1.3 Гибридные методы	7
1.2 Классификация по уровню физической достоверности	7
1.2.1 Физически обоснованные методы	8
1.2.2 Эвристические и художественные методы	11
2 Выбор алгоритма визуализации жидкости	12
2.1 Метод Stable Fluids	12
2.2 SPH (Smoothed Particle Hydrodynamics)	16
2.3 Модель мелкой воды (Shallow Water)	18
2.4 FLIP (Fluid-Implicit-Particle)	19
2.5 MLS-MPM (Material Point Method)	20
2.6 Обоснование выбора метода	21
3 Исследование алгоритма и возможностей WebGPU	22
3.1 Начало работы с WebGPU	22
3.2 Анализ алгоритма и выбор шейдеров	23
4 Реализация алгоритма	24
4.1 Инициализация симуляции	25
4.1.1 Параметры симуляции	25
4.1.2 Динамические параметры	25
4.1.3 Буферы	26
4.2 Создание шейдеров	26

5	Разработка веб-приложения для визуализации	28
5.1	Создание веб-приложения	28
5.2	Подключение WebGPU	29
6	Тестирование, оптимизация и анализ результатов	30
6.1	Отступы и выравнивание данных в буферах	30
6.2	Ping-pong буферы	32
6.3	Результат	34
	Заключение	36
	Список используемых источников	37
	Приложения	38
	Приложение А	38

Введение

Визуализация течения жидкости — одна из классических задач компьютерной графики. Впервые она была решена еще в 80-х, но тем не менее реалистичное моделирование все еще остается весьма сложной задачей, совмещающей в себе непростую теоретическую часть и большой объем математических вычислений. Стоит отметить, что под симуляцией жидкости могут понимать как гидро, так и газодинамических процессы, например вода или дым. С практической точки зрения, симуляция жидкости может быть полезна в самых разных областях — от спецэффектов в кино и игровых движков до инженерных расчётов аэродинамики и анализах атмосферных явлений. Причем в прикладных задачах важны не столько строгость физических моделей, сколько сохранение ключевых качеств течения (несжимаемость, вихревая структура, гладкость поля скоростей) при приемлемой вычислительной стоимости.

Физическая основа моделирования обычно строится на уравнениях Навье–Стокса, которые формулируют законы сохранения массы и импульса. Полное решение этих уравнений требует больших вычислительных ресурсов, поэтому в задачах интерактивной визуализации применяются устойчивые приближённые схемы, дающие адекватную визуальную достоверность при существенно меньших затратах вычислений.

С точки зрения компьютерных систем, современные методы вычислений ориентированы на перенос интенсивных расчетов на GPU, а сами симуляции зачастую разрабатываются не под конкретные ОС, а сразу под несколько. Относительно современным решением последней проблемы стала поддержка браузерами методов рендеринга на GPU, что позволяет достичь кросс-платформенной совместимости по определению. Долгое время в вебе доминировал WebGL, основанный на классическом графическом конвейере. Однако появление WebGPU изменило ситуацию. Этот

модуль предоставил более низкоуровневый доступ, унифицированные вычислительные шейдеры и явное управление ресурсами, что довольно близко по модели к Vulkan, Direct3D 12 и Metal. Это делает реализацию численных алгоритмов в браузере более эффективной и переносимой между платформами.

В данной курсовой работе рассматривается интерактивная визуализация двумерного течения жидкости с использованием WebGPU. В качестве платформы выбрано веб-приложение на TypeScript с использованием фреймворка Next.js: такой стек обеспечивает упрощенный процесс разработки самого приложения и позволяет эффективно задействовать возможности GPU через WebGPU для выполнения вычислительно сложных операций.

Цель работы — исследовать алгоритмы и разработать веб-приложение, демонстрирующее визуализацию течения жидкости в реальном времени на основе выбранного численного алгоритма. В рамках работы планируется описать физико-математические основы моделирования, обосновать выбор алгоритма и технологий, реализовать и постараться оптимизировать решение с точки зрения качества визуализации и производительности. Результатом станет интерактивное приложение, позволяющее пользователю наблюдать и взаимодействовать с моделью течения жидкости.

1. Изучение предметной области

Визуализация течения жидкости в компьютерной графике опирается на различные численные и графические методы. Для систематического анализа удобно ввести классификацию алгоритмов по двум основным основаниям:

- по **представлению среды** (эйлеровы, лагранжевы и гибридные);
- по **степени физической достоверности** (физически обоснованные и эвристические методы).

1.1. Классификация по способу представления среды

С точки зрения математического описания и хранения данных о состоянии жидкости в памяти компьютера, выделяют три основных подхода [1].

1.1.1. Эйлеровы (сеточные) методы

В эйлеровом подходе жидкость рассматривается как континуальная среда, а её состояние описывается полями величин на фиксированной пространственной сетке. На узлах или в ячейках сетки хранятся компоненты вектора скорости $u = (u_x, u_y, u_z)$, давление p , плотность ρ и другие параметры.

Численное решение уравнений движения жидкости в эйлеровом подходе сводится к последовательному обновлению значений этих полей по шагам времени.

1.1.2. Лагранжевы (частичные) методы

В лагранжевом подходе вместо фиксированной сетки рассматривается набор дискретных частиц, которые движутся одним потоком. Каждая

частица характеризуется положением x_i , скоростью v_i , массой m_i , а также, при необходимости, плотностью, давлением и другими атрибутами.

Движение частиц задаётся интегрированием уравнений движения во времени. Частицы могут интерпретироваться как материальные точки или как носители некоторого объёма жидкости. Подобные методы широко применяются, например, в алгоритмах сглаженной гидродинамики частиц (Smoothed Particle Hydrodynamics, SPH) и позиционно-ориентированной динамики (Position-Based Fluids, PBF).

1.1.3. Гибридные методы

Гибридные методы сочетают эйлеровы и лагранжевы подходы. Типичный пример — методы типа PIC/FLIP (Particle-in-Cell / Fluid-Implicit-Particle), в которых:

- сетка используется для решения глобальных задач (например, для обеспечения несжимаемости и вычисления давления);
- частицы используются для хранения массы и импульса, а также для повышения детализации (брызги, капли, мелкие вихри).

Такая комбинация позволяет одновременно воспользоваться устойчивостью и простотой сеточных схем и гибкостью частичных методов при описании сложных поверхностей и локальных эффектов.

1.2. Классификация по уровню физической достоверности

Второй важный аспект классификации — это степень, в которой алгоритм соответствует физическим законам течения жидкости.

1.2.1. Физически обоснованные методы

Физически обоснованные методы основываются на уравнениях гидродинамики и стремятся к точному или приближенному решению этих уравнений.

Численное решение уравнений Навье-Стокса является классическим подходом к моделированию течения несжимаемой жидкости [2].

Для вязкой несжимаемой жидкости ($\rho = \text{const}$) систему можно записать как:

$$\begin{cases} \frac{\partial \mathbf{u}}{\partial t} = -(\mathbf{u} \cdot \nabla) \mathbf{u} - \frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f}, \\ \nabla \cdot \mathbf{u} = 0. \end{cases} \quad (1)$$

Здесь

- $\mathbf{u}(\mathbf{x}, t)$ — поле скорости жидкости, $u = (u_x, u_y, u_z)$;
- $p(\mathbf{x}, t)$ — поле давления;
- ρ — плотность (для несжимаемой жидкости считаем постоянной);
- ν — кинематическая вязкость ($\nu = \eta/\rho$, где η — коэффициент динамической вязкости);
- $\mathbf{f}(\mathbf{x}, t)$ — вектор внешних сил на единицу массы (например, гравитация $\mathbf{g}$).

Оператор набла ∇ в декартовых координатах имеет вид

$$\nabla = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z} \right). \quad (2)$$

В зависимости от типа поля он задаёт:

- *градиент* скалярного поля $\phi(x, t)$:

$$\nabla\phi = \left(\frac{\partial\phi}{\partial x}, \frac{\partial\phi}{\partial y}, \frac{\partial\phi}{\partial z} \right); \quad (3)$$

- *дивергенцию* векторного поля u :

$$\nabla \cdot u = \frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y} + \frac{\partial u_z}{\partial z}; \quad (4)$$

- *лапласиан* (оператор диффузии) скалярного или векторного поля:

$$\nabla^2\phi = \frac{\partial^2\phi}{\partial x^2} + \frac{\partial^2\phi}{\partial y^2} + \frac{\partial^2\phi}{\partial z^2}, \quad (5)$$

а для векторного поля u лапласиан применяется покомпонентно:

$$\nabla^2 u = (\nabla^2 u_x, \nabla^2 u_y, \nabla^2 u_z).$$

Условие $\nabla \cdot u = 0$ в (1) называется уравнением непрерывности и означает несжимаемость: локально объём жидкости не меняется, масса не создаётся и не исчезает.

При численном решении системы (1) ключевую роль играет уравнение Пуассона для давления¹. Его получают, применяя оператор дивергенции к уравнению движения и используя условие несжимаемости $\nabla \cdot u = 0$. В результате для поля давления $p(x, t)$ получается эллиптическое уравнение вида:

$$\nabla^2 p = \rho \nabla \cdot (-(u \cdot \nabla)u + f), \quad (6)$$

где правая часть задаётся известными на текущем шаге скоростями и внешними силами.

Уравнение (6) определяет такое распределение давления, которое подправляет поле скорости так, чтобы после шага интегрирования по времени выполнялось условие несжимаемости. Во многих численных

¹Подробнее про уравнение Пуассона https://en.wikipedia.org/wiki/Poisson%27s_equation#Fluid_dynamics

схемах сначала вычисляют промежуточную скорость без учёта давления, затем решают уравнение Пуассона (6) и, используя найденное поле p , проецируют скорость на соленоидальное подпространство.

Метод решёточных уравнений Больцмана (LBM) является альтернативным подходом к моделированию, отличающимся от прямого численного решения уравнений Навье-Стокса. В отличие от традиционных методов вычислительной гидродинамики, работающих на макроскопическом уровне, LBM оперирует на мезоскопическом уровне, основываясь на кинетической теории газов.

Вместо непосредственного вычисления полей давления и скорости, метод моделирует эволюцию функции распределения частиц $f_i(x, t)$ на дискретной решётке. Основное уравнение LBM с оператором столкновений (часто используется приближение Бхатнагара-Гросса-Крука) выглядит следующим образом:

$$f_i(x + c_i \Delta t, t + \Delta t) = f_i(x, t) - \frac{\Delta t}{\tau} (f_i(x, t) - f_i^{eq}(x, t)), \quad (7)$$

где:

- $f_i(x, t)$ — функция распределения частиц, движущихся в направлении i ;
- c_i — дискретный набор векторов скоростей, определяемый топологией решётки (например, D2Q9 или D3Q19);
- τ — время релаксации, связанное с кинематической вязкостью жидкости;
- $f_i^{eq}(x, t)$ — локальная равновесная функция распределения.

Макроскопические величины восстанавливаются как моменты

функций распределения:

$$\rho(x, t) = \sum_i f_i(x, t), \quad \rho(x, t) u(x, t) = \sum_i f_i(x, t) c_i.$$

Показано, что в гидродинамическом пределе (при малых числах Кнудсена и Маха) из уравнения (7) с помощью разложения Чепмена—Энского восстанавливаются уравнения Навье—Стокса. Благодаря полностью локальному характеру вычислений метод особенно эффективен при реализации на графических процессорах (GPU).

Визуализация в физических методах является прямым следствием численного решения этих уравнений (или их упрощённых форм). К ним относятся: эйлеровские схемы на сетке (включая метод *stable fluids*), SPH, гибридные подходы PIC/FLIP и др. Основное достоинство таких методов — высокая физическая достоверность и предсказуемое поведение модели при изменении параметров.

1.2.2. Эвристические и художественные методы

Эвристические (или художественные) методы не стремятся к строгому решению уравнений движения жидкости. Вместо этого они используют различные визуальные приёмы, которые создают у наблюдателя впечатление течения жидкости. К таким приёмам можно отнести:

- процедурные текстуры и шумы (например, шум Перлина, *curl-noise*) для имитации турбулентности;
- экранно-пространственные эффекты (*screen-space advection*), при которых "течет" уже отрендерованное изображение;
- простые модели адвекции и диффузии для скалярных полей без строгого учёта несжимаемости.

Подобные алгоритмы, как правило, проще в реализации и дешевле по вычислительным ресурсам, что делает их популярными в реальном времени (игровые движки, веб-визуализация). Однако они не гарантируют физической корректности и предназначены прежде всего для получения визуально

2. Выбор алгоритма визуализации жидкости

В компьютерной графике для моделирования течения жидкости широко применяются различные численные методы. Выбор конкретного алгоритма зависит от требований к качеству визуализации, производительности и сложности сцены.

Как правило, от выбранного алгоритма ожидаются:

- поддержка реалистичных, но в то же время визуально выразительных и скорее красивых эффектов;
- возможность эффективной реализации на GPU в виде набора вычислительных шейдеров;
- устойчивость при достаточно крупных шагах по времени;
- разумная сложность реализации;

Рассмотрим несколько современных подходов, широко применяемых для моделирования и визуализации жидкостей.

2.1. Метод Stable Fluids

Метод Stable Fluids [3] относится к эйлеровым, сеточным методам решения уравнений Навье-Стокса для несжимаемой жидкости. Пространство дискретизуется равномерной прямоугольной сеткой; в ячейках (или

на их границах) хранятся поле скорости, давление и дополнительные скалярные поля (плотность дыма, температура, цвет и т.п.). Подход был предложен Й. Стамом специально для задач интерактивной компьютерной графики и основан на неявной, безусловно устойчивой схеме интегрирования по времени.

Исходной точкой служат уравнения Навье-Стокса для несжимаемой жидкости (1). Используя теорему о разложении Гельмгольца, векторное поле скорости $\mathbf{w}$ можно единственным образом разложить в виде:

$$\mathbf{w} = \mathbf{u} + \nabla q, \quad (8)$$

где $\nabla \cdot \mathbf{u} = 0$, а q — скалярное поле [3]. Такое разложение позволяет выделить бездивергентную (соленоидальную) часть поля и градиентную составляющую. Это приводит к возможности введения оператора P , который вычитает градиент давления и делает поле скорости несжимаемым. В компактном виде уравнение скорости записывается как

$$\frac{\partial \mathbf{u}}{\partial t} = P(-(\mathbf{u} \cdot \nabla)\mathbf{u} + \nu \nabla^2 \mathbf{u} + \mathbf{f}). \quad (9)$$

В численном алгоритме Stable Fluids один шаг по времени разбивается на несколько подшагов (operator splitting). Пусть $\mathbf{u}^n$ — поле скорости в момент времени t^n . Тогда поле $\mathbf{u}^{n+1}$ для времени $t^{n+1} = t^n + \Delta t$ получается последовательным применением следующих стадий:

1. **Добавление внешних сил.** На этом шаге учитываются внешние воздействия такие как гравитация и прочее. Предполагая, что $\mathbf{f}$ мало меняется за интервал Δt , получаем промежуточное поле

$$\mathbf{w}_1 = \mathbf{u}^n + \Delta t \mathbf{f}. \quad (10)$$

2. **Адвекция (полулагранжева схема).** Ключевая идея Stable Fluids — использовать полулагранжевую адвекцию, основанную на методе характеристик. Для каждого центра ячейки в точке $\mathbf{x}$ трассируется обратная траектория на время Δt назад по полю скорости $\mathbf{w}_1$:

$$\mathbf{x}_0 = \mathbf{x} - \Delta t \mathbf{w}_1(\mathbf{x}). \quad (11)$$

Новое значение скорости в точке $\mathbf{x}$ берётся как интерполированное значение старого поля в точке $\mathbf{x}_0$:

$$\mathbf{w}_2(\mathbf{x}) = \mathbf{w}_1(\mathbf{x}_0). \quad (12)$$

В дискретном виде это означает: для каждой ячейки выполняется обратный шаг частицы и билинейная (в 2D) или трилинейная (в 3D) интерполяция по соседним ячейкам. Такая схема безусловно устойчива и допускает довольно крупные шаги по времени, хотя и вводит численную диффузию.

3. **Диффузия (вязкость).** Вязкость описывается диффузионным слагаемым $\nu \nabla^2 \mathbf{u}$. В Stable Fluids он интегрируется неявно, что также даёт безусловную устойчивость по отношению к параметру ν . Дискретизация по времени приводит к уравнению вида

$$(I - \nu \Delta t \nabla^2) \mathbf{w}_3 = \mathbf{w}_2,$$

где I — тождественный оператор. После дискретизации лапласиана по сетке это уравнение превращается в разреженную систему линейных алгебраических уравнений, которая решается итерационными методами (Якоби, Гаусса—Зейделя, многосеточные методы и т.д.).

4. **Проекция на несжимаемое поле (решение для давления).** После

стадий адвекции и диффузии поле $\mathbf{w}_3$ вообще говоря не удовлетворяет условию $\nabla \cdot \mathbf{u} = 0$. Чтобы восстановить несжимаемость, решается уравнение Пуассона для скалярного поля давления:

$$\nabla^2 p = \rho \nabla \cdot \mathbf{w}_3,$$

после чего из скорости вычитается градиент давления:

$$\mathbf{u}^{n+1} = \mathbf{w}_3 - \frac{\Delta t}{\rho} \nabla p.$$

В дискретной форме это снова задача решения разреженной линейной системы. В случае периодических граничных условий возможно особенно эффективное решение через БПФ (FFT), так как лапласиан диагонализуется в фурье—базисе.

Для дополнительных скалярных полей (плотность дыма, температура, текстурные координаты) используется аналогичный шаблон: источник $\rightarrow$ адвекция по уже вычисленному полю скорости $\mathbf{u}^{n+1} \rightarrow$ диффузия и (при необходимости) диссипация. С точки зрения реализации на GPU каждая стадия естественно оформляется отдельным проходом или compute—шейдером над регулярной текстурой, что делает метод удобным для WebGPU и интерактивных приложений в браузере.

Для метода Stable Fluids можно выделить следующие преимущества:

- безусловная численная устойчивость, позволяющая использовать относительно крупные шаги по времени без дестабилизации решения;
- простая регулярная структура данных, хорошо подходящая для реализации на GPU (двумерные и трёхмерные текстуры, вычислительные шейдеры);

- естественная поддержка взаимодействия с внешними силами и дополнительными скалярными полями (дым, температура, текстурные координаты);
- хорошая визуальная правдоподобность для дыма, газа и двумерных симуляций воды при сравнительно низком разрешении сетки.

Однако данный подход имеет и свои ограничения. Полулагранжева адвекция и неявная диффузия вносят заметную численную диссипацию (рассеивание / dissipation), которая сглаживает мелкие вихри и детали и приводит к потере вихревости (vorticity). Для компенсации этого эффекта часто используют дополнительные приёмы вроде vorticity confinement. Строгая привязка к равномерной сетке затрудняет адаптивное увеличение разрешения в некоторых локальных областях. В трёхмерном случае же вычислительная сложность возрастает квадратично по числу ячеек в одном измерении, что делает реализацию полноценного 3D Stable Fluids слишком ресурсоёмкой.

2.2. SPH (Smoothed Particle Hydrodynamics)

Метод SPH (Smoothed Particle Hydrodynamics) [1] относится к лагранжевым частичным методам. Жидкость представляется набором частиц с массой, положением и скоростью, которые перемещаются вместе с потоком. Каждая частица интерпретируется как маленький объём жидкости, в котором усреднены плотность, давление и другие величины.

Ключевая идея SPH — заменить непрерывные поля (плотность, давление, скорость) на их приближенные (сглаженные) оценочные значения, вычисляемыми по соседним частицам. Для этого вводится сглаживающее ядро $W(\mathbf{r}, h)$ с радиусом влияния h . Плотность в точке, соответствующей

частице i , восстанавливается суммированием влияния соседей:

$$\rho_i \approx \sum_j m_j W(\mathbf{x}_i - \mathbf{x}_j, h),$$

где m_j — масса частицы j , $\mathbf{x}_j$ — её положение. Давление обычно задаётся через уравнение состояния $p = p(\rho)$, а силы давления, вязкости и внешние силы получаются из дискретизации уравнений Навье—Стокса в частично—ориентированной форме.

Численный шаг SPH обычно включает следующие этапы: обновление плотности по соседям, вычисление давления, подсчёт сил (давление, вязкость, внешние поля, взаимодействие с границами) и интегрирование уравнений движения для обновления скоростей и положений частиц. Для ускорения поиска соседей вводятся пространственные структуры данных (регулярная решетка, хеш—таблица, деревья и т.п.), так как каждая частица взаимодействует только с соседями в радиусе h .

Для метода SPH можно выделить следующие преимущества:

- естественная поддержка свободной поверхности, брызг и разрывов струи, так как частицы просто разлетаются в пространстве;
- удобная работа со сложной геометрией и подвижными твёрдыми телами: границы задаются через специальные граничные частицы или коллизии;
- хорошо расширяется на 3D, так как алгоритм естественно трёхмерен и не привязан к структурированной сетке;
- лагранжева природа алгоритма упрощает адвекцию: не нужно явно решать задачу переноса на эйлеровой сетке;

А также следующие недостатки:

- необходимость эффективного поиска соседей (дополнительные структуры данных и накладные расходы по памяти);
- повышенная вычислительная стоимость для плотных сцен и больших 2D/3D облаков частиц;
- настройка параметров сглаживания (радиус h , форма ядра, коэффициенты в уравнении состояния и вязкости) сильно влияет на стабильность и артефакты;
- сложность обеспечения строгой несжимаемости, требующая специальных вариаций SPH (WCSPH, PCISPH, DFSPH и др.);

В контексте визуализации течения жидкости для браузера SPH представляет собой привлекательный вариант благодаря простой природе частиц и наглядности, однако требует непростой реализации поиска соседей и выбора параметров, чтобы добиться устойчивой работы в реальном времени.

2.3. Модель мелкой воды (Shallow Water)

Модель мелкой воды описывает движение тонкого слоя жидкости через уравнения для высоты свободной поверхности и средних по глубине скоростей. В простейшем случае используется сетка высот, на которой решаются гиперболические уравнения баланса.

Для этой модели можно выделить следующие преимущества:

- высокая вычислительная эффективность;
- естественная визуализация волн на поверхности (например, вода в резервуаре, волны в океане);
- простая реализация на регулярной сетке и на GPU.

И следующие недостатки:

- модель предполагает малую глубину и не описывает сложные 3D эффекты;
- трудно получить вихревую структуру потока внутри объёма жидкости;
- ограниченная физическая точность в случаях, выходящих за рамки предположений модели.

Для задач визуализации волн на плоскости модель мелкой воды является очень привлекательной, но менее универсальной, чем общий решатель уравнений Навье Стокса.

2.4. FLIP (Fluid-Implicit-Particle)

Метод FLIP сочетает Эйлерову сетку и лагранжевы частицы. Сетка используется для решения уравнений давления и несжимаемости, а частицы несут массу и импульс. На каждом шаге:

- величины переносятся с частиц на сетку;
- на сетке решается задача проекции скорости;
- обновлённая скорость интерполируется обратно к частицам;

Метод FLIP обладает рядом преимуществ. Во-первых, он обеспечивает меньшую численную диффузию по сравнению с чисто сеточными схемами, что позволяет лучше сохранять мелкие вихри и детали потока. Во-вторых, FLIP удачно сочетает стабильность вычислений с высокой степенью детализации, делая его подходящим для реалистичного моделирования жидкостей. То есть этот метод хорошо подходит для создания анимаций воды с брызгами и сложными поверхностными эффектами, однако

требует сложной реализации, а также значительных вычислительных ресурсов и памяти для хранения как частиц, так и сеточных данных.

2.5. MLS-MPM (Material Point Method)

Метод MLS-MPM можно рассматривать как развитие частично-матричных подходов и, в частности, как дальнейшее развитие идей SPH, где непрерывная среда представляется набором частиц. Как и в SPH, материальные точки (частицы) хранят массу и скорость, однако в MLS-MPM они дополнительно несут информацию о деформации, а взаимодействие между частицами осуществляется не напрямую, а через вспомогательную эйлерову сетку.

Базовый MPM (Material Point Method) представляет материал в виде набора материальных точек, которые сами по себе не сталкиваются друг с другом, а обмениваются импульсом и силами через фоновую решётку. В большинстве вариантов реализации MLS-MPM для переноса величин между частицами и узлами сетки используется метод наименьших квадратов, что повышает точность и устойчивость схемы при сильных деформациях.

Данный подход получил широкую известность благодаря использованию MPM для моделирования снега в фильме Frozen студии Disney, что наглядно демонстрирует способность метода правдоподобно описывать широкий спектр материалов — от сыпучих сред до вязких жидкостей[4].

Ключевыми преимуществами MLS-MPM являются: единый формализм для жидкостей, упругих и пластичных материалов, хорошая устойчивость при сильных деформациях и высокая визуальная реалистичность сложных эффектов (снег, грязь, вязкие жидкости). В то же время метод отличается существенной математической и алгоритмической сложностью, его сложнее эффективно реализовать в браузере, да и по отношению к цели простой демонстрации течения несжимаемой жидкости он часто оказыва-

ется избыточным.

2.6. Обоснование выбора метода

С точки зрения баланса между качеством визуализации, сложностью реализации и производительностью в браузере наиболее подходящим является сеточный метод класса Stable Fluids.

- Алгоритм напрямую реализует упрощённые уравнения Навье Стокса для несжимаемой жидкости, что позволяет связать визуализацию с физической моделью.
- Регулярная сетка и локальные операции хорошо ложатся на модель вычислительных шейдеров WebGPU.
- Метод относительно прост для реализации, включает ограниченное число стадий (адвекция, диффузия, проекция), которые удобно разделить на отдельные шейдеры.
- Получаемые эффекты (вихревые структуры, плавные завихрения дыма и жидкости) визуально выразительны и заметны пользователю.

Модели мелкой воды и частичные методы (SPH, FLIP, MLS —MPM) представляют собой интересные альтернативы и могут быть рассмотрены как направления дальнейшего расширения проекта. Однако в рамках данной курсовой работе основным выбран Эйлеров сеточный метод Stable Fluids, реализуемый на WebGPU.

3. Исследование алгоритма и возможностей WebGPU

3.1. Начало работы с WebGPU

Поскольку WebGPU является относительно новой технологией, для начала работы с ней потребуется выполнить несколько шагов по настройке [5].

Во-первых, необходимо убедиться, что браузер поддерживает WebGPU. На момент написания этой работы, поддержка WebGPU была доступна в последних версиях браузеров Chromium при включении экспериментальных функций [6].

Во-вторых, необходимо настроить графический конвейер для выполнения вычислений на GPU. В WebGPU это включает создание устройства GPU, выделение буферов для хранения данных и написание вычислительных шейдеров на языке WGSL (WebGPU Shading Language).

В-третьих, потребуется интегрировать динамические параметры симуляции, такие как скорость течения, вязкость и проч., а также эмулировать внешние силы, чтобы обеспечить интерактивную и наглядную визуализацию течения жидкости.

В результате этих шагов будет создана основа для реализации численного алгоритма моделирования жидкости с использованием WebGPU. Примерный план выполнения программы данных будет выглядеть следующим образом:

1. Инициализация WebGPU

- (a) Создание и настройка модулей WebGPU.
- (b) Создание буферов для хранения данных симуляции на GPU.
- (c) Подключение шейдеров для выполнения вычислений.

- (d) Создания графического конвейера для визуализации результатов.
- (e) Запуск цикла рендеринга и обновления симуляции.

2. Основной цикл симуляции

- (a) Обновление входных данных симуляции на CPU.
- (b) Передача этих данных на GPU через буферы WebGL.
- (c) Выполнение вычислительных шейдеров, обновления состояния симуляции на GPU.

3.2. Анализ алгоритма и выбор шейдеров

Как упоминалось уже ранее, метод Stable Fluids основан на приближении (9) уравнения Навье-Стокса. Выбирая Δt , мы можем построить симуляцию с помощью разложения на четыре этапа: добавление силы, адвекция, диффузия и проекция. Каждый из этих этапов может быть реализован с помощью отдельного шейдера, что позволяет просто и эффективно использовать вычисления на GPU.

В классическом варианте метода Stable Fluids[3] каждый этап можно записать следующим образом:

$$\mathbf{w}_0(\mathbf{x}) \xrightarrow{\text{сила}} \mathbf{w}_1(\mathbf{x}) \xrightarrow{\text{адвекция}} \mathbf{w}_2(\mathbf{x}) \xrightarrow{\text{диффузия}} \mathbf{w}_3(\mathbf{x}) \xrightarrow{\text{проекция}} \mathbf{w}_4(\mathbf{x}) \quad (13)$$

$$\mathbf{w}_1(\mathbf{x}) = \mathbf{w}_0(\mathbf{x}) + \Delta t \mathbf{f}(\mathbf{x}),$$

$$\mathbf{w}_2(\mathbf{x}) = \text{Advect}(\mathbf{w}_1, \Delta t),$$

$$\mathbf{w}_3(\mathbf{x}) = \text{Diffuse}(\mathbf{w}_2, \nu, \Delta t),$$

$$\mathbf{w}_4(\mathbf{x}) = \mathcal{P}(\mathbf{w}_3(\mathbf{x})),$$

где каждый из операторов может быть реализован с помощью отдельного шейдера. Например, адвекция может быть реализована с помощью обратного трассирования частиц, диффузия — с помощью метода Якоби, а проекция — с помощью решения уравнения Пуассона.

Также весьма полезно будет реализовать дополнительные шейдеры для визуализации объектов симуляции, а также результатов симуляций такие как отрисовка векторного поля скорости или визуализация давления.

4. Реализация алгоритма

В начале необходимо определить детали реализации, такие как размер рабочей сетки и основные параметры, которые будут использоваться в вычислениях.

Пусть у нас будет простая двумерная квадратная сетка размером $N \times N$. Длина стороны каждой ячейки равна L . Следуя классическому варианту алгоритма каждая ячейка содержит определенные в ее центре вектора $u_0(x, y)$ и $u_1(x, y)$ [7]. В каждый момент времени мы будем вычислять новые значения $u_1(x, y)$ на основе предыдущего значения $u_0(x, y)$, используя четыре этапа алгоритма Stable Fluids.

То есть мы можем представить векторное поле как два двумерных массива размером $N \times N$, где каждый элемент массива содержит вектор в соответствующей ячейке сетки.

Таким образом, мы представляем нашу симуляцию как набор буферов, хранящих данные о скорости и других физических величинах для каждой ячейки сетки. Эти буферы будут использоваться в вычислительных шейдерах для выполнения каждого из этапов алгоритма.

4.1. Инициализация симуляции

4.1.1. Параметры симуляции

Определим основные константы:

- Размерность: $N_{DIM} = 2$. 2D симуляция.
- Начало координат: $O = (0, 0)$. Центр области отрисовки.
- Физический размер пространства: $L = (750, 750)$ в пикселях. Область отрисовки
- Число ячеек: $N_x = N_y = N = 250$.
- Размер ячейки: $\Delta x = \Delta y = L_x/N_x = L_y/N_y = 3$.
- Шаг по времени: $\Delta t = 1/60$ s. 60 кадров в секунду.
- Итерации решения давления: `PRESSURE_ITERATIONS` = 20.
- Итерации решения вязкости: `VISCOSITY_ITERATIONS` = 10.
- Размер группы (для шейдера): `WORKGROUP_SIZE` = 16. $16 \times 16 = 256$ потоков.

4.1.2. Динамические параметры

Для обеспечения интерактивности симуляции, мы будем использовать динамические параметры, которые могут быть изменены пользователем во время выполнения симуляции.

Определим начальные значения динамических параметров симуляции:

- Вязкость: $\nu = \text{VISCOSITY} = 0.005$.
- Рассеивание: `DISSIPATION` = 0.999.

- Коэффициент завихренности: $\text{VORTEX_STRENGTH} = 25$.
- Гравитация (ось y): $\text{GRAVITY} = -20$.
- Количество добавляемого вещества: $\text{FAUCET_DYE} = 200$.
- Позиция мыши: $\text{MOUSE_POS} = (N_x/2, N_y/2)$.
- Скорость мыши: $\text{MOUSE_VEL} = (0, 0)$.

Для передачи этих параметров на GPU, используются uniform буферы, которые позволят нам передавать эти значения в шейдеры на каждом кадре симуляции.

4.1.3. Буферы

Для удобства реализации на GPU, мы будем использовать буферы для хранения данных сетки, такие как буфер скорости, буфер давления и буфер вещества-краски (dye). Каждый из этих буферов будет представлен в виде пар массивов, который может быть эффективно обработан параллельно на GPU при помощи техник ring-pong (переключение между двумя буферами для чтения и записи). Также для решения уравнения Пуассона потребуется буфер для дивергенции.

4.2. Создание шейдеров

Для реализации каждого этапа алгоритма Stable Fluids (13), были созданы отдельные вычислительные шейдеры на языке WGSL.

Ниже приведен полный список шейдеров, используемых в реализации симуляции жидкости:

00_header.wgsl Общие определения, константы и функции, подключаемые в другие шейдеры.

- 01_render.wgsl** Шейдер вывода: преобразует данные симуляции в финальный цвет для экрана.
- 11_add_force.wgsl** Добавляет внешние силы в поле скоростей (input velocity += force).
- 12_vortex_force.wgsl** Применяет вихревые силы для сохранения завихрений в потоке.
- 13_gravity.wgsl** Моделирует действие гравитации на плотность/скорость среды.
- 21_advect_vel.wgsl** Адвекция скорости — перенос значений скорости по самому полю.
- 31_viscosity.wgsl** Учет вязкости: сглаживание и диффузия скоростей.
- 41_clear_pressure.wgsl** Сбрасывает или инициализирует поле давления перед итерациями решения.
- 42_divergence.wgsl** Вычисляет дивергенцию скоростного поля для контроля несжимаемости.
- 43_pressure_jacobi.wgsl** Итеративный решатель давления методом Якоби для устранения дивергенции.
- 44_subtract_gradient.wgsl** Корректирует скорости вычитанием градиента давления (проекция на несжимаемое поле).
- 51_add_dye.wgsl** Добавляет краску/плотность в поле (внешние источники плотности).
- 52_splat_dye.wgsl** Локальные всплески краски (splat) — быстрые импульсные добавления плотности.

53_advect_dye.wgsl Адвекция краски по скоростному полю (перенос плотности).

Заметим, что первая цифра в названии шейдера указывает на этап алгоритма, а вторая — на конкретную операцию внутри этого этапа. Такой подход упрощает организацию и понимание структуры шейдеров.

Этап 0 — рендеринг, этапы 1-4 — основные этапы алгоритма Stable Fluids, этап 5 — работа с краской (dye).

5. Разработка веб-приложения для визуализации

Разработанное приложение, основанное на WebGPU, также требует определенных шагов по настройке и инициализации. В этой секции описывается процесс создания веб-приложения с использованием TypeScript и Next.js, а также интеграция WebGPU для выполнения вычислений на GPU.

5.1. Создание веб-приложения

Для удобства разработки и скорости разработки веб-приложения был выбран фреймворк Next.js, который предоставляет мощные инструменты для создания приложений на TypeScript.

В начале необходимо настроить проект Next.js с поддержкой TypeScript. Это включает в себя создание нового проекта, установку необходимых зависимостей и настройку конфигурационных файлов, линтера и прочих инструментов разработки рекомендованных в официальной документации Next.js [8].

Листинг 1. Создание нового проекта Next.js с TypeScript

```
1 pnpm create next-app@latest my-app --yes
2 cd my-app
3 pnpm dev
```

После создания проекта, необходимо установить типы для WebGPU, чтобы обеспечить поддержку TypeScript при работе с API WebGPU. Кроме того, для создания пользовательского интерфейса с динамическими параметрами симуляции будет использоваться библиотека `lil-gui`.

Листинг 2. Установка зависимостей

```
1 pnpm add -D @webgpu/types # Установка типов для WebGPU
2 pnpm add lil-gui          # Установка библиотеки lil-gui
```

5.2. Подключение WebGPU

Для веб-приложения были разработаны React-компоненты, один из которых является точкой входа для инициализации WebGPU и создания контекста рендеринга. В этом компоненте происходит вызов функций инициализации WebGPU, а также передача параметров мыши (координаты и скорость) для удобного взаимодействия с симуляцией.

Листинг 3. Часть React-компонента для инициализации WebGPU

```
1 useEffect(() => {
2   const canvas = canvasRef.current
3   if (!canvas) return
4
5   setupMouseListener(canvas, params.mouse)
6
7   // This will hold the cleanup function returned by main()
8   let cleanup: (() => void) | undefined
9
10  main(canvas, params)
11    .then((c) => {
12      // Store the cleanup function for later
13      cleanup = c
```

```

14     })
15     .catch((err) => {
16         console.error(err)
17     })
18
19     return () => {
20         if (cleanup) cleanup()
21     }
22 }, []) // only initialize WebGPU once
23 }

```

Функция `main` отвечает за инициализацию WebGPU, создание устройства GPU, выделение буферов и настройку вычислительных шейдеров. Она также запускает основной цикл рендеринга и обновления симуляции.

Таким образом, веб-приложение на Next.js соединяется с WebGPU, позволяя эффективно использовать возможности GPU для выполнения вычислительных задач, связанных с симуляцией жидкости.

6. Тестирование, оптимизация и анализ результатов

По ходу тестирования было обнаружено множество багов и недочетов. Все они были исправлены, что позволило улучшить качество и правдоподобность симуляции. Тем не менее итоговая производительность оказалась ниже ожидаемой, поэтому также были проведены улучшения и общая оптимизация, направленные на повышение скорости работы программы.

6.1. Отступы и выравнивание данных в буферах

Во время тестирования были выявлены проблемы с передачей через буферы данных между шейдерами WGSL и TypeScript.

Листинг 4. Пример передачи динамических параметров из CPU на GPU

```

1 // render.ts
2
3 const data = new Float32Array([
4     params.dt,
5     params.viscosity,
6     params.diffusion,
7     params.dissipation,
8     N_CELLS.x,
9     params.vorticity,
10    params.gravity,
11    params.faucet_dye
12 ])
13 device.queue.writeBuffer(uniformBuffer, 0, data.buffer)

```

Листинг 5. Пример получения динамических параметров в шейдере

```

1 // 00_header.wgsl
2
3 @group(1) @binding(0) var<uniform> sim_uniforms : SimUniforms;
4 struct SimUniforms {
5     delta_t: f32,
6     viscosity: f32,
7     diffusion: f32,
8     dissipation: f32,
9
10    N: f32,          // assume square grid N x N
11    vorticity: f32,
12    gravity: f32,
13    faucet_dye: f32,
14 };

```

Ключевым моментом было правильное выравнивание данных в uniform буферах, что позволило избежать неправильных результатов при чтении данных в шейдерах. Поскольку чтение осуществляется в формате vec4 (по 4 компоненты), необходимо было точно упаковать данные и учитывать выравнивание. В этом случае использовалось кратное 4 количество float-значений в uniform буфере из-за не потребовалось добавлять дополнительные пустые значения.

6.2. Ping-pong буферы

Во время тестирования производительности симуляции было выявлено, что использование техники ping-pong для буферов значительно улучшает скорость выполнения шейдеров. Эта техника позволяет эффективно переключаться между двумя буферами для чтения и записи данных, что минимизирует задержки и повышает общую производительность.

Листинг 6. Пример использования ping-pong буферов для решения давления

```
1 // render.ts
2 // 30) clear pressure field
3 cpass.setPipeline(pipelines.clearPressurePipeline)
4 cpass.setBindGroup(1, bindGroups.simBG[p])
5 cpass.dispatchWorkgroups(wx, wy)
6 p ^= 1 // IMPORTANT: now the cleared pressure lives in p_in of the new
    state
7
8 // 31) divergence (writes div)
9 cpass.setPipeline(pipelines.divergePipeline)
10 cpass.setBindGroup(1, bindGroups.simBG[p])
11 cpass.dispatchWorkgroups(wx, wy)
12 // p stays the same
13
14 // 32) pressure solve (Jacobi)
15 for (let k = 0; k < PRESSURE_ITERATIONS; k++) {
16     cpass.setPipeline(pipelines.pressurePipeline)
17     cpass.setBindGroup(1, bindGroups.simBG[p])
18     cpass.dispatchWorkgroups(wx, wy)
19     p ^= 1 // CRITICAL: swaps p_in/p_out so next iter reads the new
        pressure
20 }
21
22 // 33) subtract gradient (updates velocity using pressure)
23 cpass.setPipeline(pipelines.gradientPipeline)
24 cpass.setBindGroup(1, bindGroups.simBG[p])
25 cpass.dispatchWorkgroups(wx, wy)
26 p ^= 1
```


Тем не менее, реализация ping-pong буферов потребовала тщательной настройки и управления состояниями буферов, во время использования которых были выявлены некоторые сложности. В частности, необходимо было обеспечить правильное управление состояниями буферов, чтобы избежать конфликтов при чтении и записи данных. Это включало в себя тщательное отслеживание текущего состояния буферов и правильное переключение между ними в нужные моменты времени (см. листинг 6).

Листинг 7. Подключение буферов и решение давления методом Якоби в WGSL

```

1 // 00_header.wgsl
2
3 // Velocity ping-pong
4 @group(1) @binding(0) var<storage, read> vel_in : array<vec2<f32>>;
5 @group(1) @binding(1) var<storage, read_write> vel_out :
    array<vec2<f32>>;
6 // Dye ping-pong
7 @group(1) @binding(2) var<storage, read> dye_in : array<f32>;
8 @group(1) @binding(3) var<storage, read_write> dye_out : array<f32>;
9
10 // Projection buffers
11 @group(1) @binding(4) var<storage, read_write> divergence : array<f32>;
12 @group(1) @binding(5) var<storage, read> p_in : array<f32>;
13 @group(1) @binding(6) var<storage, read_write> p_out : array<f32>;
14
15 // 42_pressure_jacobi.wgsl
16
17 // divergence + p_in -> p_out (one Jacobi iteration)
18 @compute @workgroup_size(16,16,1)
19 fn cs_main(@builtin(global_invocation_id) gid: vec3<u32>) {
20     let x = i32(gid.x);
21     let y = i32(gid.y);
22     if (x < 0 || y < 0 || x >= i32(N_u32()) || y >= i32(N_u32())) {
23         return; }
24
25     let i = idx(u32(x), u32(y));
26
27     // pass-through vel/dye so ping pressure iterations doesn't

```

```

        scramble them
27  vel_out[i] = vel_in[i];
28  dye_out[i] = dye_in[i];
29
30  let pl = p_at(x - 1, y);
31  let pr = p_at(x + 1, y);
32  let pb = p_at(x, y - 1);
33  let pt = p_at(x, y + 1);
34
35  let div = divergence[i];
36  p_out[i] = (pl + pr + pb + pt - div) * 0.25;
37 }

```

В листинге 7 можно увидеть как шейдер решения давления и использует ring-pong буферы, переключая состояние буферов (в листинге 6) после каждой итерации. Это позволяет эффективно использовать результаты предыдущей итерации для следующей, что значительно ускоряет процесс решения.

6.3. Результат

Текущая реализация выполнена с физической точки зрения достоверна и не имеет графических багов. Она демонстрирует корректность и стабильность работы симуляции. Примеры финальной симуляции и поля давления приведены на рисунках 6.1 и 6.2.

На рисунке 6.1 представлена визуализация двумерного течения жидкости, демонстрирующая динамическое поведение потока и взаимодействие с добавляемым веществом (краской). На 5 шаге конвейера происходит добавление краски в центр области симуляции, что позволяет наблюдать за распространением вещества в потоке жидкости. На систему действует гравитация, вектор течения же направлен в правый верхний угол, что видно по форме и направлению завихрений в жидкости.

На рисунке 6.2 показано распределение давления в области симуля-

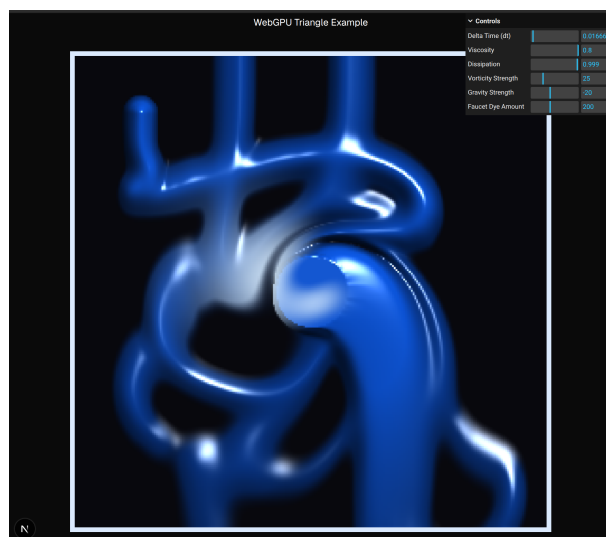


Рис. 6.1. Течение жидкости

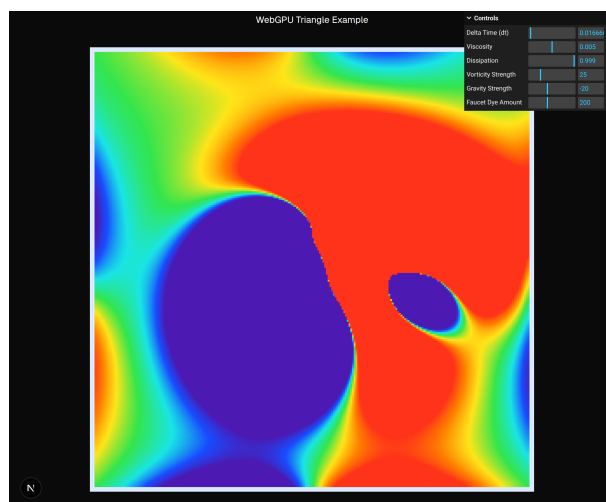


Рис. 6.2. Поле давления

ции. От синего (нулевого) к красному (максимальному) что позволяет оценить влияние различных сил и условий на поведение жидкости. Исходя из направления течения жидкости, можно наблюдать, как давление изменяется в зависимости от скорости и направления потока.

Заключение

В ходе выполнения курсовой работы была рассмотрена задача интерактивной визуализации двумерного течения жидкости с использованием современных веб технологий. Были изучены физико-математические основы моделирования гидро- и газодинамических процессов, а также некоторые алгоритмы визуализации модели жидкости. На основе исследования был выбран и реализован алгоритм Stable Fluids, который обеспечивает баланс между качеством визуализации, вычислительной эффективностью и сложностью реализации.

В работе был использован подход к реализации симуляции жидкости на графическом процессоре с использованием вычислительных шейдеров. Это позволило эффективно обрабатывать большие массивы данных, описывающие поле скоростей и скалярные величины, и тем самым достичь большей производительности. Было уделено внимание правильной настройке графического и вычислительного конвейера, организации данных в виде буферов, и разделению этапов симуляции на отдельные, вычислительные шаги, которые следовали алгоритму Stable Fluids.

Результатом работы стало интерактивное веб-приложение, демонстрирующее визуализацию течения жидкости в реальном времени и позволяющее пользователю взаимодействовать с симуляцией. Это подтверждает, что современные браузерные технологии позволяют эффективно взаимодействовать с GPU и решать задачи визуализации, которые ранее были бы реализовывали только в качестве нативных приложений.

Список используемых источников

- [1] Sinuo Liu iaokun Wang, Yanrui Xu. Physics-based fluid simulation in computer graphics: Survey, research trends, and challenges. 2024. URL <https://webpace.science.uu.nl/~telea001/uploads/PAPERS/CVMJ23/paper.pdf>.
- [2] Robert Bridson. Fluid Simulation for Computer Graphics. 2013. URL <https://github.com/mordak42/fluid-simulation/blob/master/doc/Fluid%20Simulation%20for%20Computer%20Graphics%2C%20Second%20Edition.pdf>.
- [3] Jos Stam. Stable fluids. 1999. URL https://pages.cs.wisc.edu/~chaol/data/cs777/stam-stable_fluids.pdf.
- [4] Webgpu fluid simulations: High performance and real-time rendering, . URL <https://tympanus.net/codrops/2025/02/26/webgpu-fluid-simulations-high-performance-real-time-rendering/>.
- [5] Webgpu fundamentals, . URL <https://webgpufundamentals.org/>.
- [6] Статус реализации webgpu, . URL <https://github.com/gpuweb/gpuweb/wiki/Implementation-Status>.
- [7] Jos Stam. Real-time fluid dynamics for games. 2003. URL <https://graphics.cs.cmu.edu/nsp/course/15-464/Fall09/papers/StamFluidforGames.pdf>.
- [8] Next.js Documentation. URL <https://nextjs.org/docs>.

Приложения

Приложение А

Исходный код реализации симуляции жидкости доступен по ссылке:

<https://github.com/sorrtory-vercel/stable-fluids-webgpu>.